ECE3622
Embedded System Design

Han Wang

Electrical and Computer Engineering

Temple University

Agenda

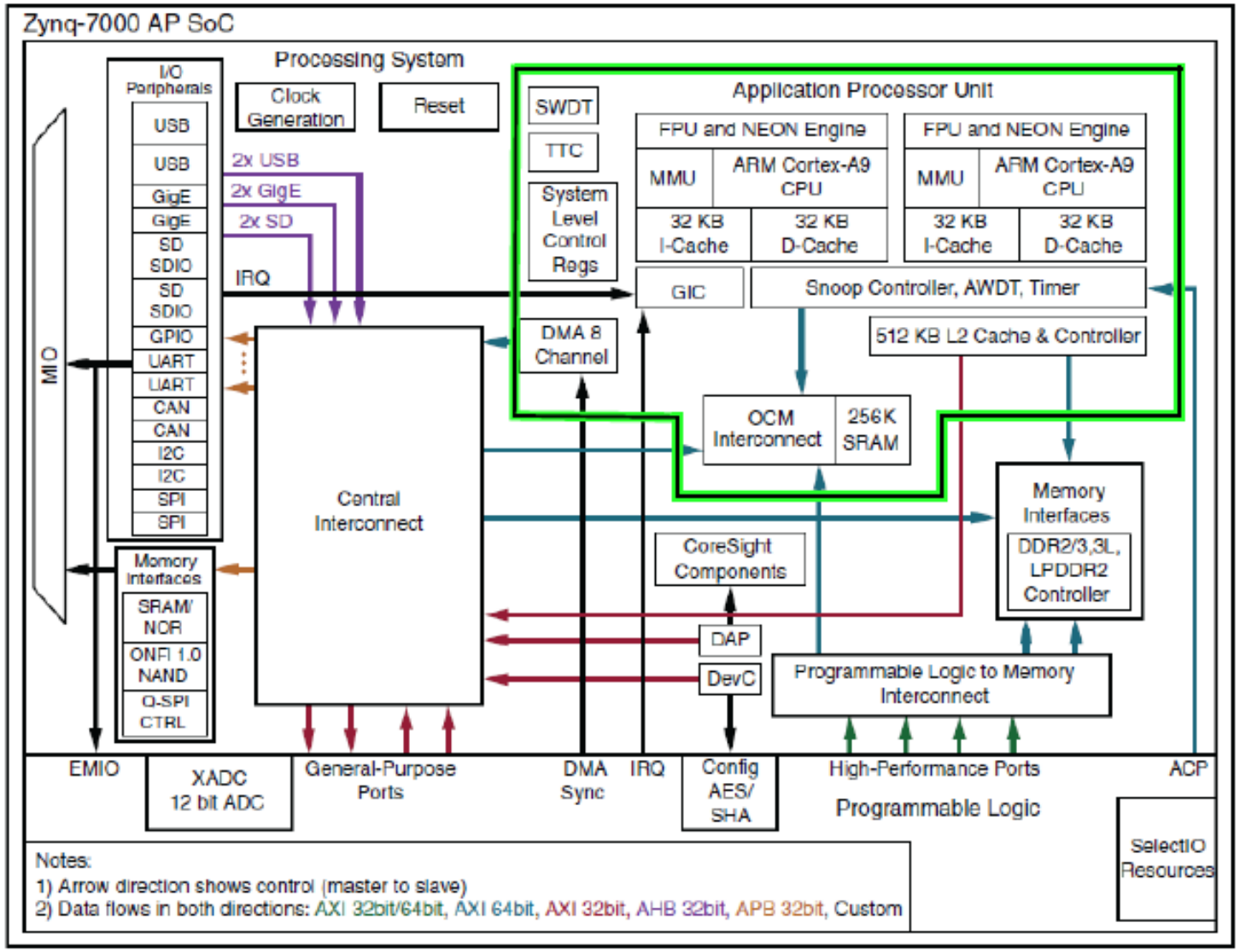
- Zynq Book Tutorials:
  Lab 1: Exercises: 1A, 1B, 1C2A,
- GPIO
- Demo

ZYBO Board Components

Callout	Component Description	Callout	Component Description
1	Power Switch	15	Processor Reset Pushbutton
2	Power Select Jumper and battery header	16	Logic configuration reset Pushbutton
3	Shared UART/JTAG USB port	17	Audio Codec Connectors
4	MIO LED	18	Logic Configuration Done LED
5	MIO Pushbuttons (2)	19	Board Power Good LED
6	MIO Pmod	20	JTAG Port for optional external cable
7	USB OTG Connectors	21	Programming Mode Jumper
8	Logic LEDs (4)	22	Independent JTAG Mode Enable Jumper
9	Logic Slide switches (4)	23	PLL Bypass Jumper
10	USB OTG Host/Device Select Jumpers	24	VGA connector
11	Standard Pmod	25	microSD connector (Reverse side)
12	High-speed Pmods (3)	26	HDMI Sink/Source Connector
13	Logic Pushbuttons (4)	27	Ethernet RJ45 Connector
14	XADC Pmod	28	Power Jack

Source: ZYBO Reference Manual

The Zynq Processing System

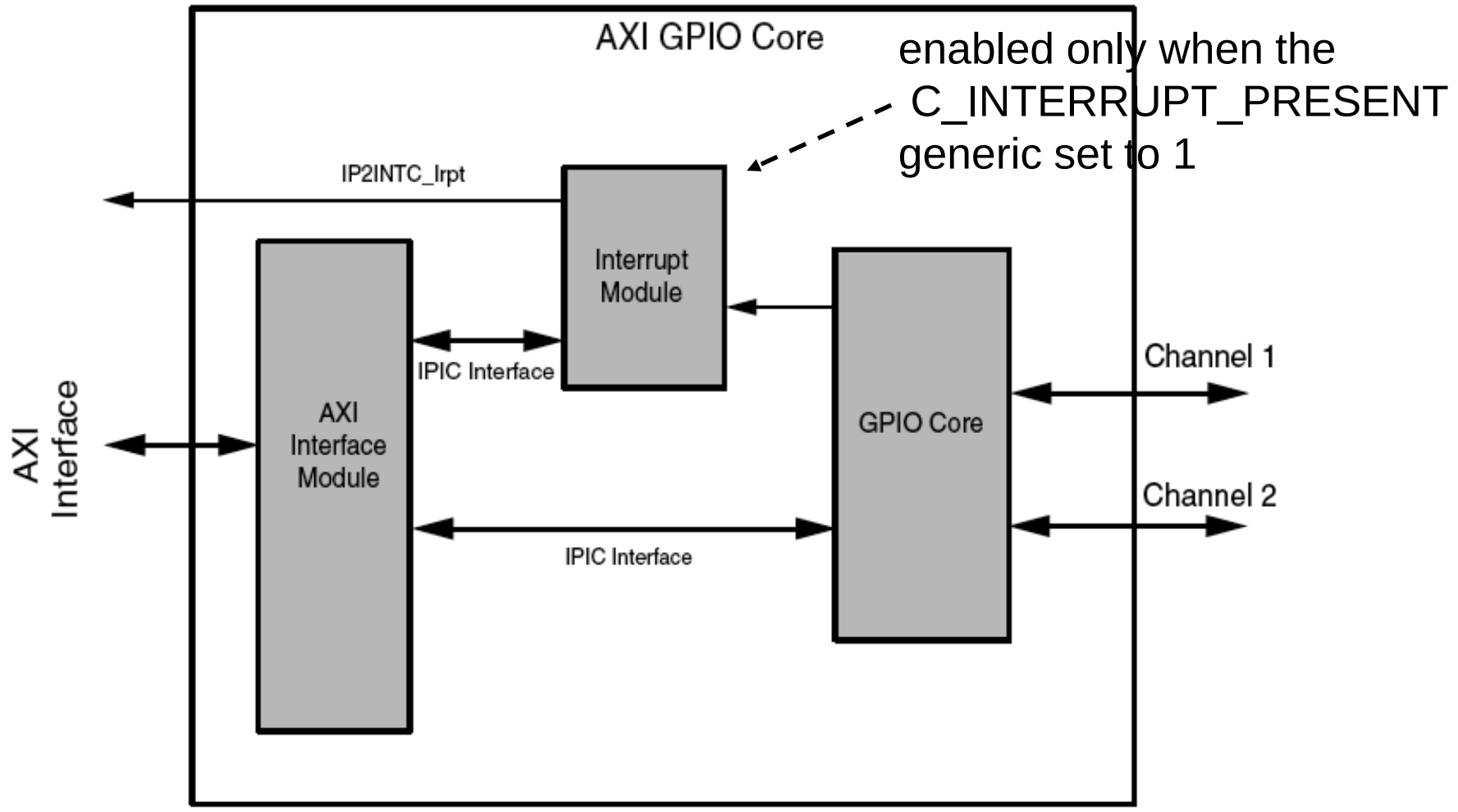


Source: The Zynq Book

AXI GPIO Features

- Configurable single or dual GPIO channel(s)
- Each channel configurable to have from 1 to 32 bits
- Dynamic programming of each GPIO bit as input or output
- Individual configuration of each channel
- Independent reset values for each bit of all registers
- Optional interrupt request generation
- AXI4-Lite interface to Processing System

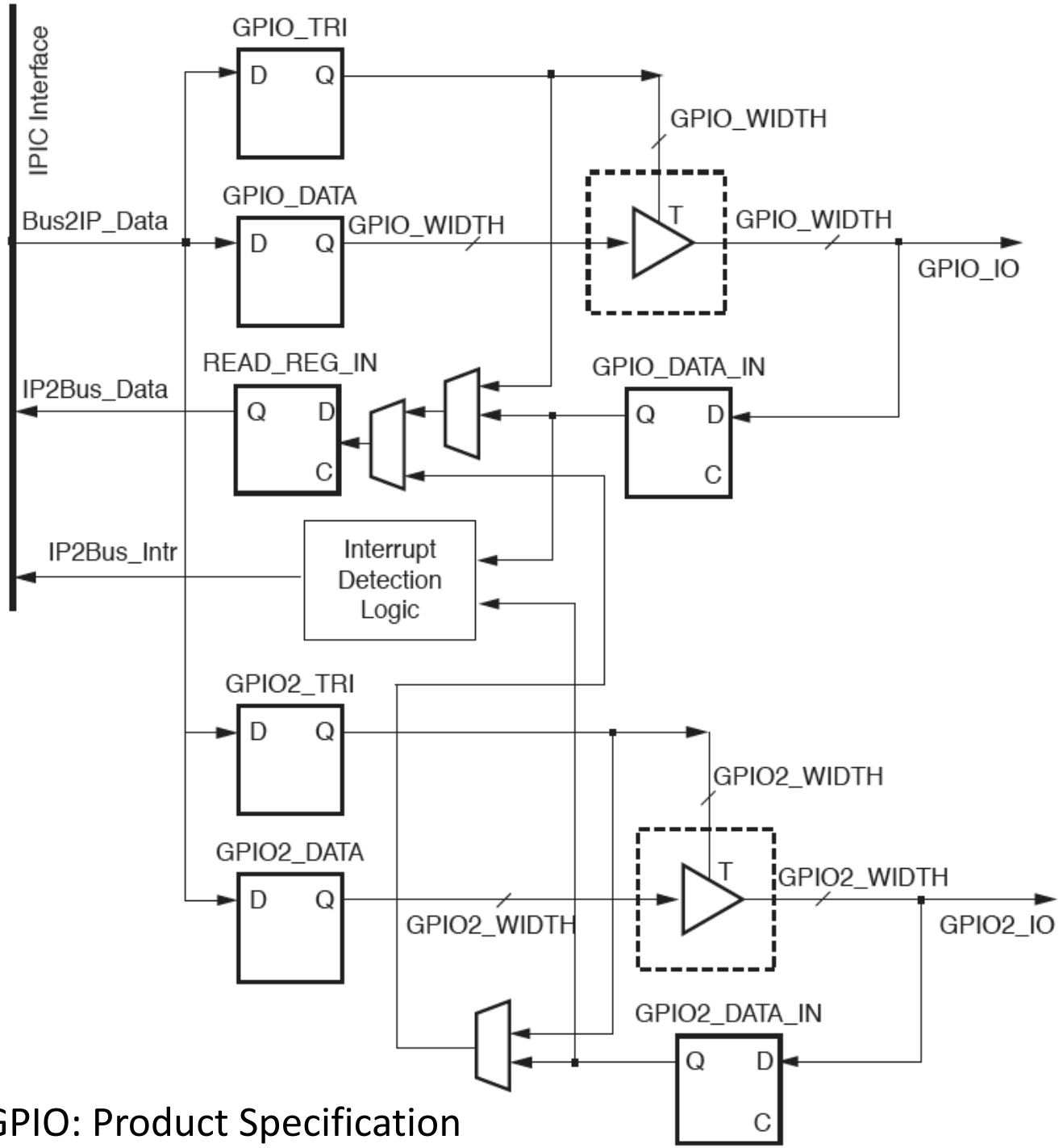
Block Diagram of AXI GPIO



IPIC – IP Interconnect interface

Source: LogiCORE IP AXI GPIO: Product Specification

GPIO Core



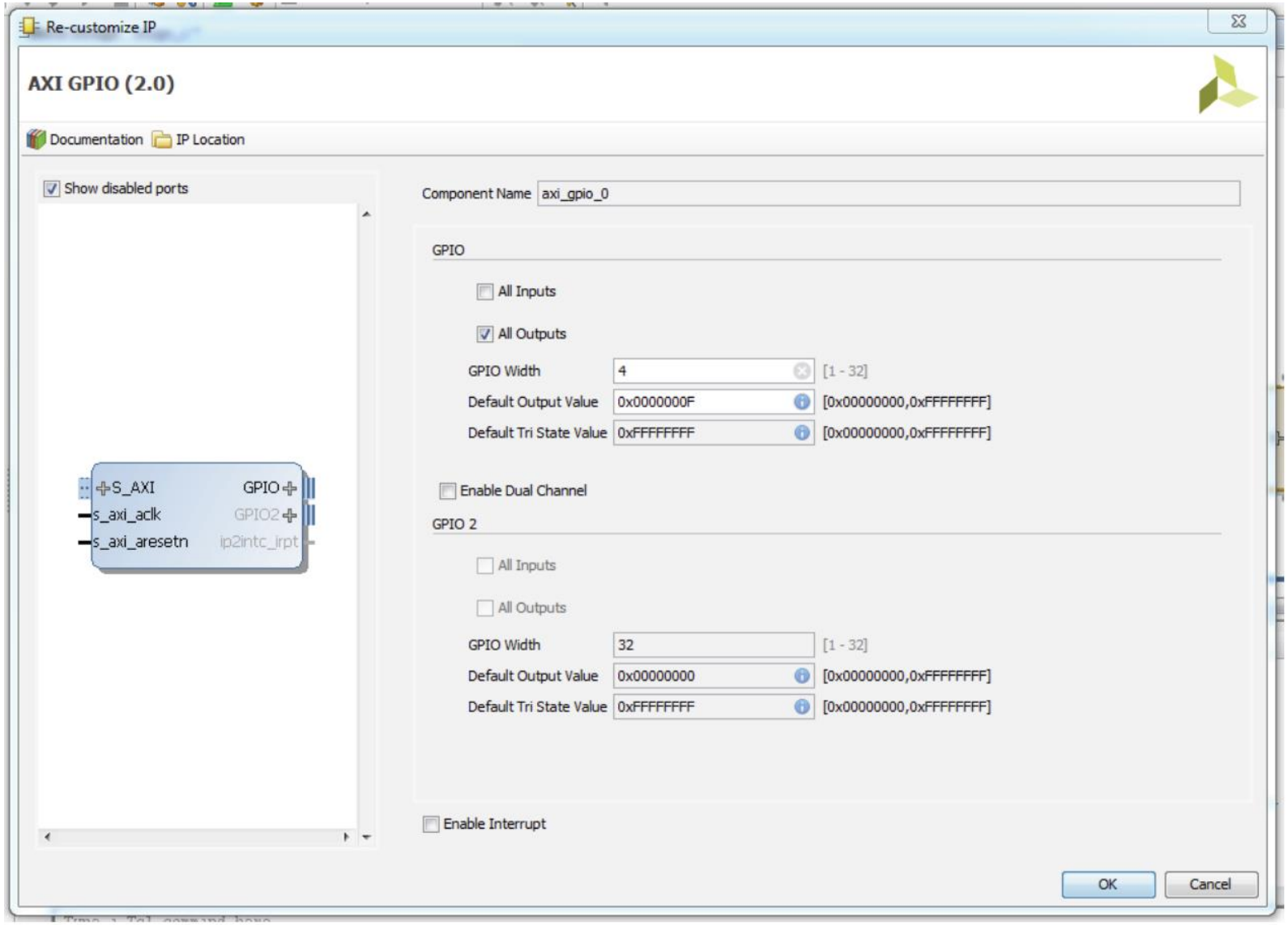
Source: LogiCORE IP AXI GPIO: Product Specification

GPIO Core Parameters

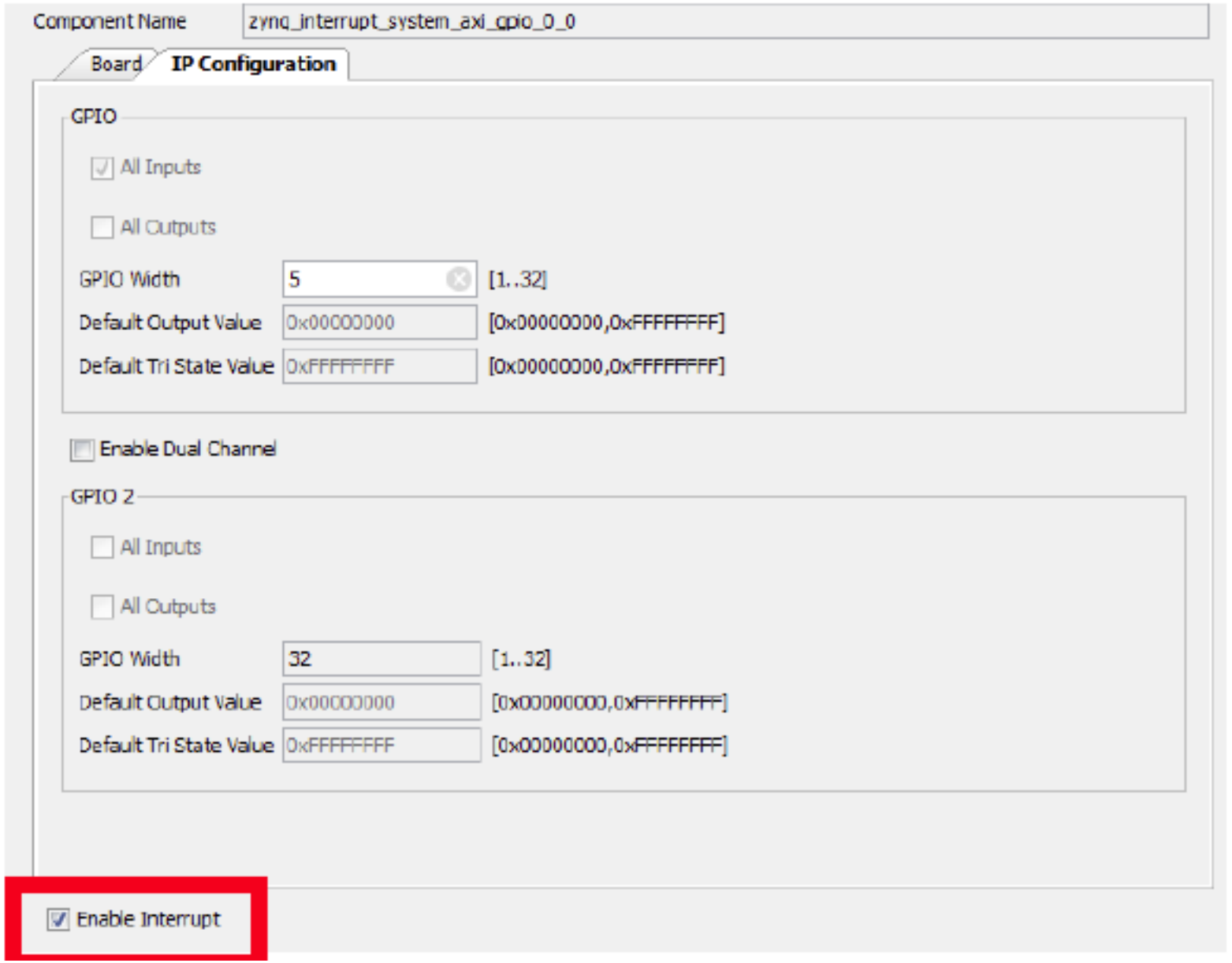
GPIO Parameters					
G7	GPIO Channel 1 Data Bus Width	C_GPIO_WIDTH	1–32	32	integer
G8	GPIO Channel 2 Data Bus Width	C_GPIO2_WIDTH	1–32	32	integer
G9	AXI GPIO Interrupt	C_INTERRUPT_ PRESENT	0 = Interrupt Controller module is not present 1 = Interrupt Controller module is present	0	integer
G10	GPIO_DATA Reset Value	C_DOUT_DEFAULT	Any valid std_logic_vector	0x00000000	std_logic_ vector
G11	GPIO_TRI Reset Value	C_TRI_DEFAULT	Any valid std_logic_vector	0xFFFFFFFF	std_logic_ vector
G12	Use Dual Channel	C_IS_DUAL	0 = Single channel is enabled 1 = Both channels are enabled	0x0	integer
G13	GPIO2_DATA Reset Value	C_DOUT_DEFAULT_2	Any valid std_logic_vector	0x00000000	std_logic_ vector
G14	GPIO2_TRI Reset Value	C_TRI_DEFAULT_2	Any valid std_logic_vector	0xFFFFFFFF	std_logic_ vector

Source: LogiCORE IP AXI GPIO: Product Specification

Setting GPIO Core Parameters in Vivado



Setting GPIO Core Parameters in Vivado

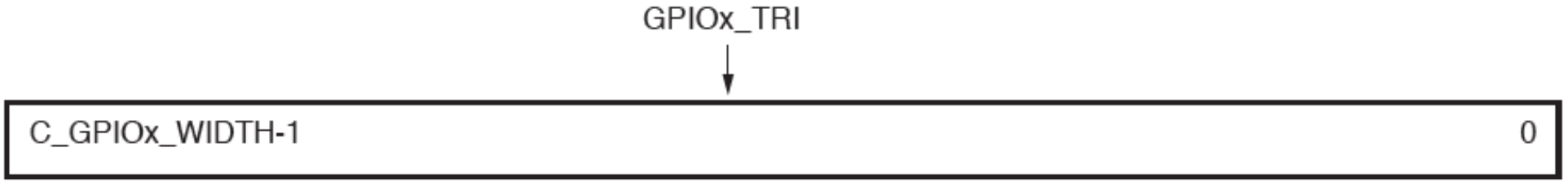


GPIO_DATA and GPIO2_DATA Registers

Bits	Name	Core Access	Reset Value	Description
C_GPIOx_WIDTH - [1:0]	GPIOx_DATA	Read/Write	C_DOUT_DEFAULT C_DOUT_DEFAULT_2	AXI GPIO Data. For each I/O bit programmed as input: • **R**: Read value on input pin. • **W**: No effect. For each I/O bit programmed as output: • **R**: No effect. • **W**: Writes value to corresponding AXI GPIO data register bit and output pin.

Source: LogiCORE IP AXI GPIO: Product Specification

GPIO_TRI and GPIO2_TRI Registers



Bits	Name	Core Access	Reset Value	Description
C_GPIOx_WIDTH - [1:0]	GPIOx_TRI	Read/Write	C_TRI_DEFAULT C_TRI_DEFAULT_2	AXI GPIO 3-state Control. Each I/O pin of the AXI GPIO is individually programmable as an input or output. For each of the bits: • 0 = I/O pin configured as output. • 1 = I/O pin configured as input.

Source: LogiCORE IP AXI GPIO: Product Specification

AXI GPIO System Parameters

System Parameter					
G1	Target FPGA family	C_FAMILY	artix7, virtex7, kintex7, spartan6, virtex6	virtex6	string
AXI Parameters					
G2	AXI Base Address	C_BASEADDR	Valid Address(1)	0xFFFFFFFF(2)	std_logic_vector
G3	AXI High Address	C_HIGHADDR	Valid Address(1)	0x00000000(2)	std_logic_vector
G4	AXI Address Bus Width	C_S_AXI_ADDR_WIDTH	32	32	integer
G5	AXI Data Bus Width	C_S_AXI_DATA_WIDTH	32	32	integer
G6	AXI interface type	C_S_AXI_PROTOCOL	AXI4LITE	AXI4LITE	string

C_HIGHADDR – C_BASEADDR ≥ 0xFFF

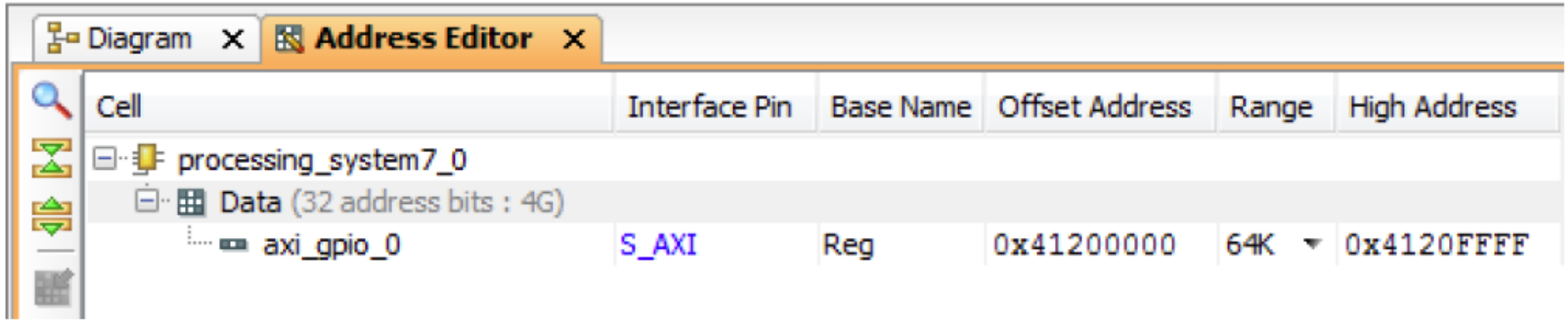
Source: LogiCORE IP AXI GPIO: Product Specification

Addresses of AXI GPIO Registers

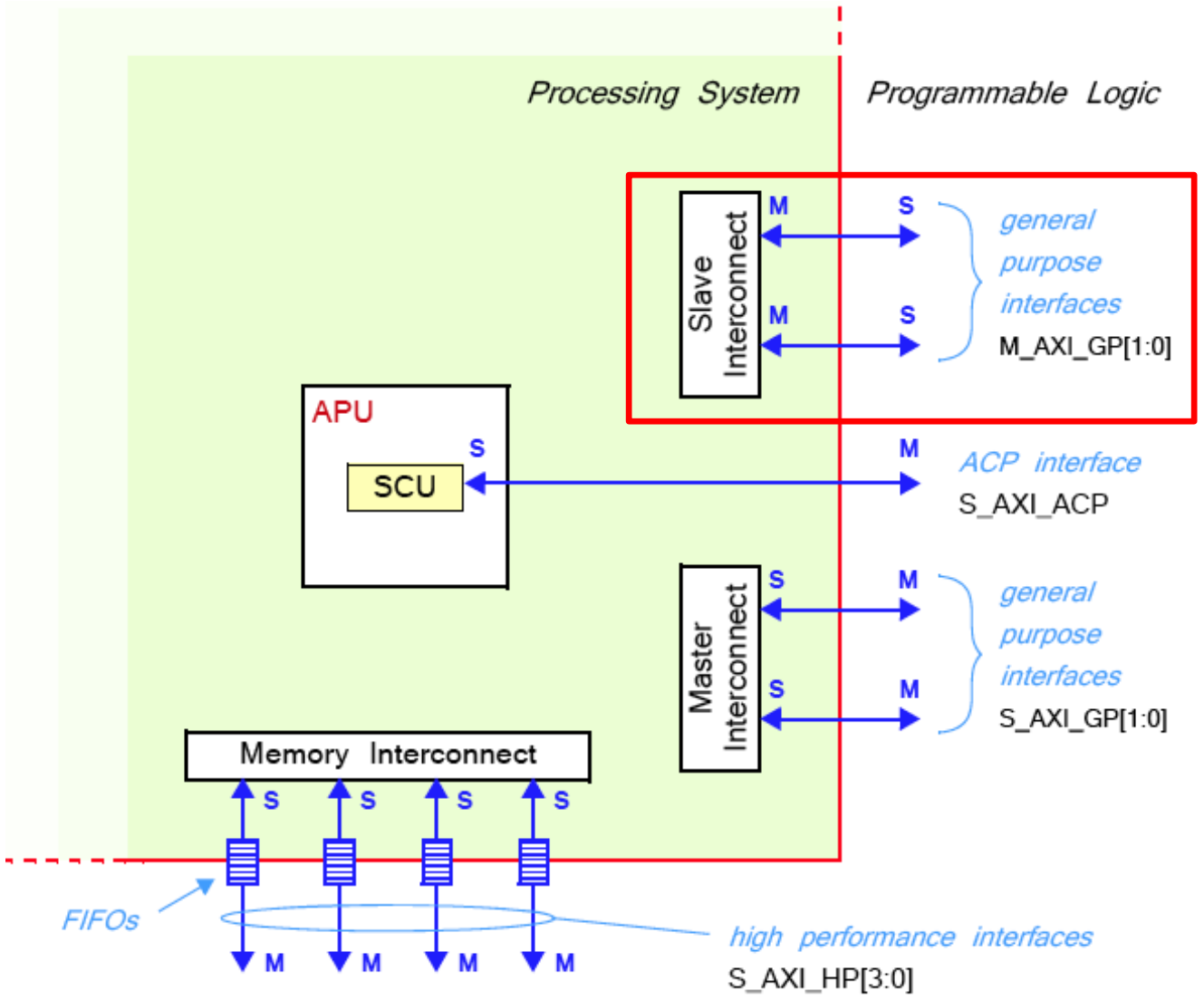
Base Address + Offset (hex)	Register Name	Access Type	Default Value (hex)	Description
C_BASEADDR + 0x00	GPIO_DATA	Read/Write	0x0	Channel 1 AXI GPIO Data Register
C_BASEADDR + 0x04	GPIO_TRI	Read/Write	0x0	Channel 1 AXI GPIO 3-state Register
C_BASEADDR + 0x08	GPIO2_DATA	Read/Write	0x0	Channel 2 AXI GPIO Data Register
C_BASEADDR + 0x0C	GPIO2_TRI	Read/Write	0x0	Channel 2 AXI GPIO 3-state Register

Source: LogiCORE IP AXI GPIO: Product Specification

Address Editor in Vivado



AXI Interconnects and Interfaces



Source: The Zynq Book

xparameters.h

```
/* Definitions for driver GPIO */
#define XPAR_XGPIO_NUM_INSTANCES 1

/* Definitions for peripheral AXI_GPIO_0 */
#define XPAR_AXI_GPIO_0_BASEADDR 0x41200000
#define XPAR_AXI_GPIO_0_HIGHADDR 0x4120FFFF
#define XPAR_AXI_GPIO_0_DEVICE_ID 0
#define XPAR_AXI_GPIO_0_INTERRUPT_PRESENT 0
#define XPAR_AXI_GPIO_0_IS_DUAL 0
```

C Program (1)

```
/* Include Files */

#include "xparameters.h"
#include "xgpio.h"
#include "xstatus.h"
#include "xil_printf.h"

/* Definitions */
#define GPIO_DEVICE_ID  XPAR_AXI_GPIO_0_DEVICE_ID
/* GPIO device that LEDs are connected to */
#define LED 0x03                        /* Initial LED value */
#define LED_DELAY 10000000              /* Software delay length */
#define LED_CHANNEL 1                   /* GPIO port for LEDs */
#define printf xil_printf               /* smaller, optimized printf */
```

C Program (2)

```
XGpio Gpio;                /* GPIO Device driver instance */

int LEDOutputExample(void)
{

   volatile int Delay;
   int Status;
   int led = LED;  /* Hold current LED value. Initialize to LED definition */

    /* GPIO driver initialization */
   Status = XGpio_Initialize(&Gpio, GPIO_DEVICE_ID);
   if (Status != XST_SUCCESS) {
            return XST_FAILURE;
   }

   /*Set the direction for the LEDs to output. */
   XGpio_SetDataDirection(&Gpio, LED_CHANNEL, 0x00);
```

C Program (3)

```
    /* Loop forever blinking the LED. */
    while (1) {
        /* Write output to the LEDs. */
        XGpio_DiscreteWrite(&Gpio, LED_CHANNEL, led);

        /* Flip LEDs. */
       led = ~led;

        /* Wait a small amount of time so that the LED blinking is visible. */
        for (Delay = 0; Delay < LED_DELAY; Delay++); }

    return XST_SUCCESS;  /* Ideally unreachable */
}
```

C Program (4)

```
/* Main function. */
int main(void){

        int Status;

        /* Execute the LED output. */
        Status = LEDOutputExample();
        if (Status != XST_SUCCESS) {
                xil_printf("GPIO output to the LEDs failed!\r\n");
        }

        return 0;
}
```